

Automation with Crontab Theory (1 hour):

Scheduling tasks using crontab. Use cases for automation in Linux. Practical (1 hour):

Automate backups of critical directories.

Task: Submit a backup configuration and log output.

TASKS

1. List 3 real-life scenarios where automating tasks with crontab would be useful on a Linux server, and write one line explaining the benefit for each.

Scenario 1

Daily Database Backup

Benefit: Automatically backs up the database every night to prevent data loss.

Scenario 2

Automatic Log Cleanup

Benefit: Deletes old log files regularly to save disk space.

Scenario 3

System Updates

Benefit: Keeps the Linux server updated with the latest security patches automatically

2. Write a crontab entry that runs a script called update_playlist.sh every day at 2:00 AM to automatically update your music playlist backup, and explain what each part of the crontab line means.

Create a Cron Job

Step 1: Open Crontab

crontab -e

Step 2: Add this line

0 2 * * * /home/du/update_playlist.sh

Save:

Esc

:wq

Explanation

- 0 = Minute
- 2 = Hour (2 AM)
- * = Every day of month
- * = Every month
- * = Every day of week

```
[root@localhost ~]# crontab -e
crontab: installing new crontab
[root@localhost ~]#
```

**3. Create a shell script named `backup_instagram_photos.sh` that copies all files from `/home/yourusername/InstagramPhotos` to `/home/yourusername/Backups/InstagramPhotos` with today's date in the backup folder name.

Hint: Use the `date` command to generate a folder like `InstagramPhotos_2024-06-10`.**

Create Backup Script

Step 1: Create Source Folder

```
mkdir -p /home/du/InstagramPhotos
```

Step 2: Create Sample Files

```
touch /home/du/InstagramPhotos/photo1.jpg
```

```
touch /home/du/InstagramPhotos/photo2.jpg
```

```
touch /home/du/InstagramPhotos/photo3.jpg
```

Step 3: Create Backup Folder

```
mkdir -p /home/du/Backups
```

Step 4: Create Script

```
vi /home/du/backup_instagram_photos.sh
```

Press i

Paste:

```
#!/bin/bash
```

```
TODAY=$(date +%Y-%m-%d)
```

```
mkdir -p /home/du/Backups/InstagramPhotos_$TODAY
```

```
cp -r /home/du/InstagramPhotos/*  
/home/du/Backups/InstagramPhotos_$TODAY/
```

Press

Esc

:wq

Step 5: Give Execute Permission

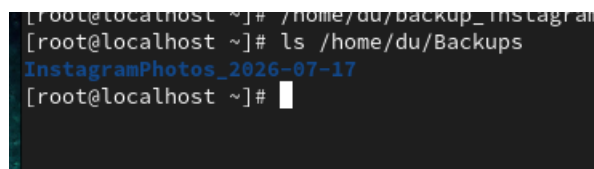
```
chmod +x /home/du/backup_instagram_photos.sh
```

Step 6: Test Script

```
/home/du/backup_instagram_photos.sh
```

Step 7: Verify

```
ls /home/du/Backups
```



```
[root@localhost ~]# /home/du/backup_instagram_photos.sh  
[root@localhost ~]# ls /home/du/Backups  
InstagramPhotos_2026-07-17  
[root@localhost ~]#
```

4. Schedule your backup_instagram_photos.sh script to run every Sunday at 11:30 PM using crontab, and redirect both standard output and errors to a log file named backup_log.txt in your home directory.

Schedule Script

Open crontab

crontab -e

Add:

```
30 23 * * 0 /home/du/backup_instagram_photos.sh >> /home/du/backup_log.txt  
2>&1
```

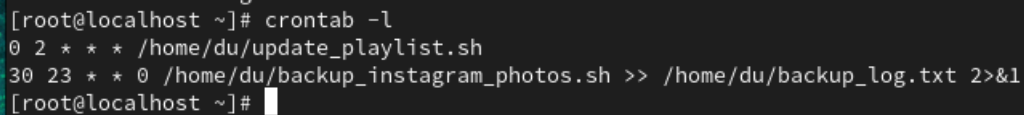
Save

Esc

:wq

Verify

crontab -l



```
[root@localhost ~]# crontab -l  
0 2 * * * /home/du/update_playlist.sh  
30 23 * * 0 /home/du/backup_instagram_photos.sh >> /home/du/backup_log.txt 2>&1  
[root@localhost ~]#
```

5. After your backup runs, open backup_log.txt and submit the last 5 lines as proof that your backup and logging worked.
Hint: Use the tail command to view the last lines of your log.

Open the script:

vi /home/du/backup_instagram_photos.sh

Replace it with:

```
#!/bin/bash
```

```
TODAY=$(date +%Y-%m-%d)
```

```
echo "Backup started at $(date)"
```

```
mkdir -p /home/du/Backups/InstagramPhotos_$TODAY
```

```
cp -r /home/du/InstagramPhotos/*  
/home/du/Backups/InstagramPhotos_$(date +%Y-%m-%d)/
```

```
echo "Backup completed successfully."
```

Save:

Esc

:wq

Make it executable again:

```
chmod +x /home/du/backup_instagram_photos.sh
```

Run it:

```
/home/du/backup_instagram_photos.sh >> /home/du/backup_log.txt 2>&1
```

View the last 5 lines

```
tail -5 /home/du/backup_log.txt
```

```
[root@localhost ~]# tail -5 /home/du/backup_log.txt  
Backup started at Fri Jul 17 12:10:42 PM IST 2026  
Backup completed successfully.  
[root@localhost ~]#
```

Verify Backup

```
ls -R /home/du/Backups
```

```
[root@localhost ~]# ls -R /home/du/Backups  
/home/du/Backups:  
InstagramPhotos_2026-07-17  
  
/home/du/Backups/InstagramPhotos_2026-07-17:  
photo1.jpg photo2.jpg photo3.jpg  
[root@localhost ~]#
```